Estrutura de dados - ADS

Conteúdo da aula anterior ?

Funções

Centro Paula Souza

Revisando aula 06

Funções

Em C++, **uma função é um bloco de código** que realiza uma tarefa específica. **Funções são usadas para dividir um programa em partes menores** e mais gerenciáveis, o que **torna o código mais organizado**, legível e reutilizável.

As funções em C++ têm várias características importantes:

Funções

Nome da Função: Toda função em C++ tem um nome que a identifica. O nome é usado para chamar a função quando você deseja executar o código dentro dela.

Parâmetros: Uma função pode receber zero ou mais parâmetros, que são valores ou variáveis que você passa para a função quando a chama. Os parâmetros permitem que a função receba dados de fora e os processe internamente.

Tipo de Retorno: Toda função em C++ possui um tipo de retorno que especifica o tipo de dado que a função retorna após a execução. Isso pode ser um tipo primitivo, um objeto personalizado ou até mesmo "void" se a função não retornar nenhum valor.

Corpo da Função: O corpo da função é o bloco de código entre as chaves {} que define o que a função faz. É onde você escreve as instruções para realizar uma tarefa específica.

Chamada de Função: Para executar o código dentro de uma função, você a chama pelo seu nome, fornecendo os argumentos necessários (se houver) entre parênteses. A função é executada, e o valor de retorno (se houver) pode ser usado em outras partes do programa.

Sintaxe

```
#include <iostream>

// Declaração da função
tipo_de_retorno nome_da_funcao(parâmetros) {
    // Corpo da função
    // Instruções para realizar a tarefa
    return valor_de_retorno; // Opcional, dependendo do tipo de retorno
}

int main() {
    // Chamada da função
    tipo_de_retorno resultado = nome_da_funcao(argumentos);

    // Resto do programa

    return 0;
}
```

Exemplo de função sem retorno

```
#include <iostream>
using namespace std;

void imprimirMensagem() {
    cout << "Olá, Mundo!" << endl;
}

int main() {
    imprimirMensagem();
    return 0;
}
```

Exemplo função sem retorno

```
#include <iostream>
using namespace std;

void dobrarPorReferencia(int &numero) {
    numero *= 2;
}

int main() {
    int valor = 5;
    dobrarPorReferencia(valor);

    cout << "O valor dobrado é: " << valor << endl;

    return 0;
}
```

Exemplo função sem retorno

```
#include <iostream>
using namespace std;

void lerEntradaUsuario(int &valor) {
    cout << "Digite um número: ";
    cin >> valor;
}

int main() {
    int numero;
    lerEntradaUsuario(numero);

    cout << "Você digitou: " << numero << endl;

    return 0;
}
```

Exemplo com retorno

```
#include <iostream>
using namespace std;

// Declaração da função
int soma(int a, int b) {
    int resultado = a + b;
    return resultado;
}

int main() {
    // Chamada da função
    int x = 5;
    int y = 3;
    int resultadoSoma = soma(x, y);

    cout << "A soma de " << x << " e " << y << " e: " << resultadoSoma << endl;

    return 0;
}
```

Exemplo com retorno (completar)

```
#include <iostream>
using namespace std;

double calcularMedia(double numero1, double numero2) {


}

int main() {
    double valor1 = 5.5;
    double valor2 = 7.0;

    double resultado = calcularMedia(valor1, valor2);

    cout << "A média de " << valor1 << " e " << valor2 << " é: " << resultado << endl;

    return 0;
}
```

Desafio

```
#include <iostream>
using namespace std;

bool ehPar(int numero) {

}

int main() {
    int numero = 7;

    if (                ) {
        cout << numero << " é um número par." << endl;
    } else {
        cout << numero << " não é um número par." << endl;
    }

    return 0;
}
```

Correção Desafio

```
#include <iostream>
using namespace std;

bool ehPar(int numero) {
    return numero % 2 == 0;
}

int main() {
    int numero = 7;

    if (ehPar(numero)) {
        cout << numero << " é um número par." << endl;
    } else {
        cout << numero << " não é um número par." << endl;
    }

    return 0;
}
```

Agenda

Aula de hoje

- Programação Orientada a objeto

AGENDA

Prof. Adriano Daniel

Centro Paula Souza

O que é orientação a objetos ?

Programação Orientada a Objetos (POO):

A **POO é um paradigma que se concentra na modelagem de objetos**, que são instâncias de classes. As classes definem **estruturas de dados** e **comportamentos associados**. O código é organizado em **torno de objetos** que se comunicam entre si, através de troca de mensagens.

Por que programar utilizando esse conceito?

Motivos para programar em POO

- ✓ Modelagem do Mundo Real
- ✓ Reutilização de Código
- ✓ Código organizado e estruturado
- ✓ Segurança
- ✓ Flexibilidade
- ✓ Manutenção Simplificada
- ✓ Escalabilidade

Linguagens de programação que implementam esse paradigma:

Então, o que é uma classe ?

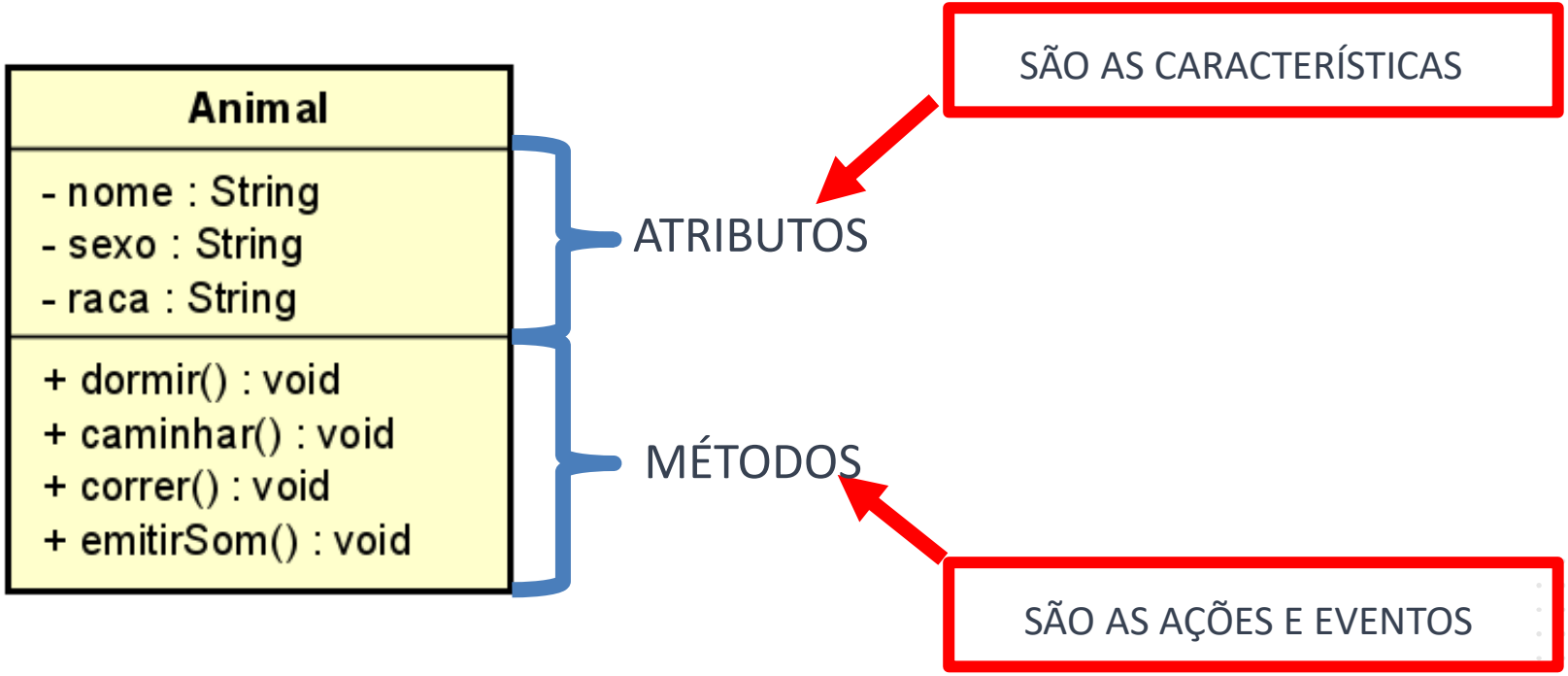
CLASSES

É um **molde ou modelo para criar os objetos**, ainda, pode ser considera uma **coleção de objetos.**

CLASSE

OBJETOS

EXEMPLO DE CLASSE (UML)



Certo, e o Objeto ?

OBJETO

É o biscoito feito a partir do **molde (Classe)**

OBJETOS

EXEMPLO DE CLASSE / OBJETO (UML)

CLASSE

Animal

- nome : String
- sexo : String
- raca : String

+ dormir() : void
+ caminhar() : void
+ correr() : void
+ emitirSom() : void

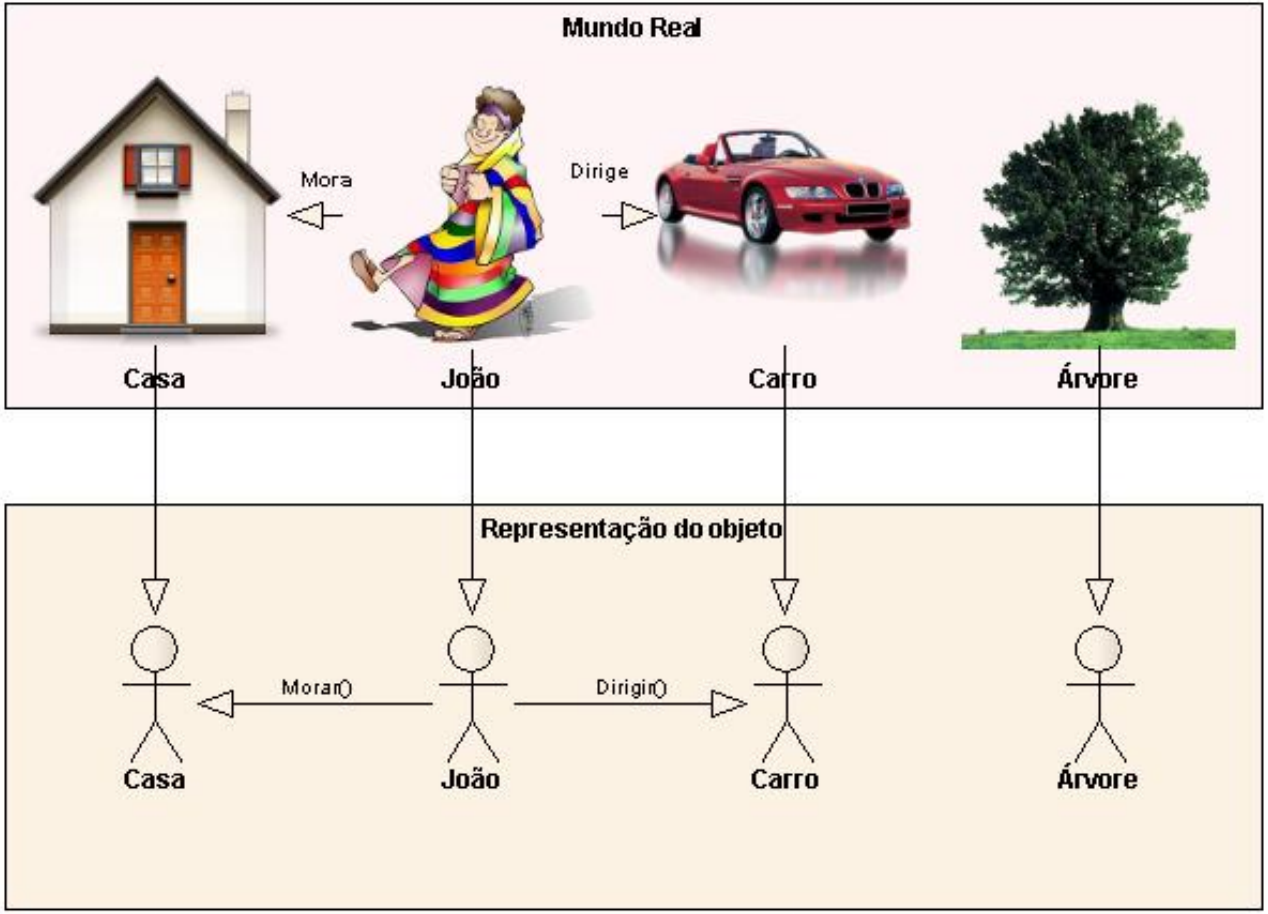
OBJETOS

CACHORRO
FEMEA
BEAGLE
6 HORAS
A CADA 2HORAS
10 KM/H
LATIR

GATO
MACHO
PERSA
6 HORAS
A CADA 3 HORAS
12 KM/H
MIAR

PILARES DO POO

A **abstração** envolve a simplificação e a representação de objetos do mundo real em um contexto de programação. Ela foca nos aspectos relevantes e ignora os detalhes desnecessários.


Mundo Real
Mora
Dirige
Casa
João
Carro
Árvore
Representação do objeto
Morar()
Dirigir()
Casa
João
Carro
Árvore

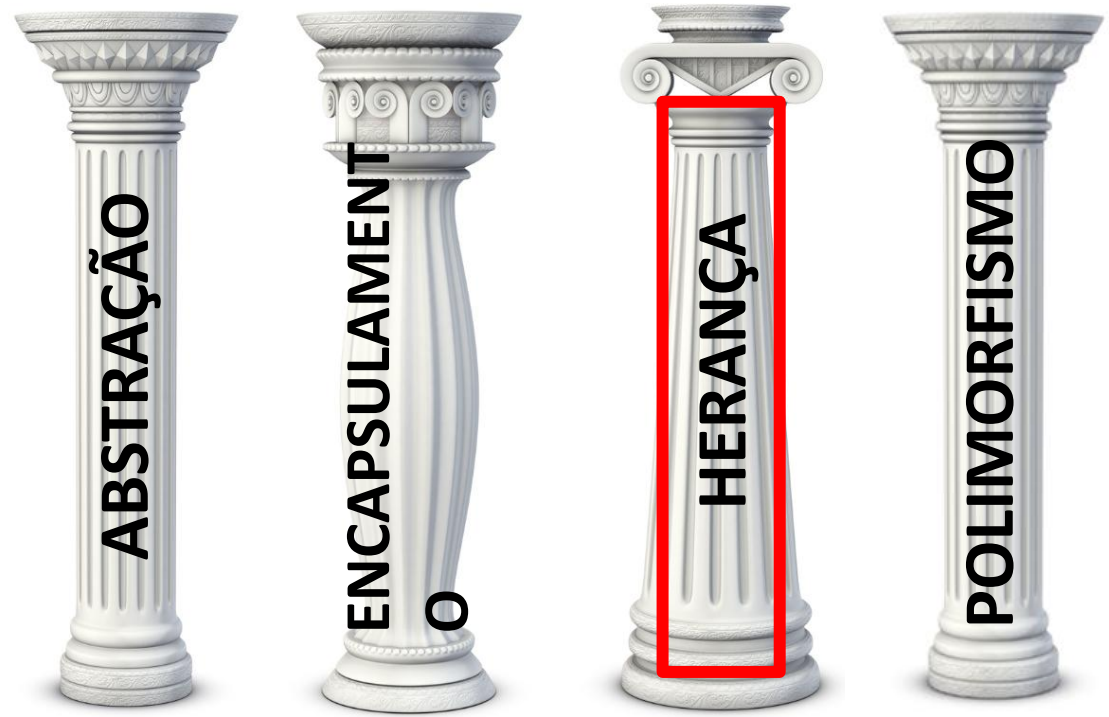
PILARES DO POO

O **encapsulamento** envolve a combinação de dados (atributos) e os métodos (comportamentos) que operam nesses dados em uma única unidade, a classe. **O encapsulamento protege os dados internos de uma classe**, permitindo que eles sejam acessados e modificados apenas por meio dos métodos definidos

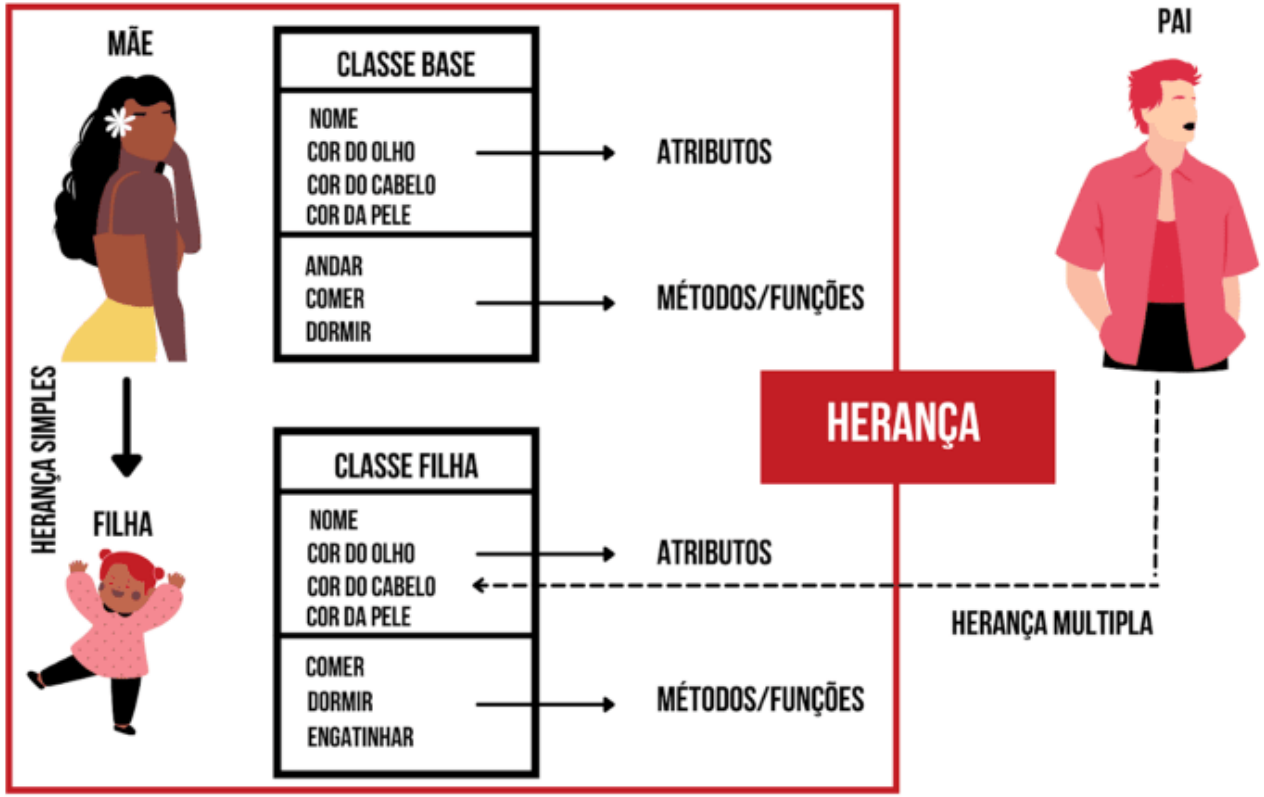
EXEMPLO: ENCAPSULAMENTO

EXEMPLO: ENCAPSULAMENTO

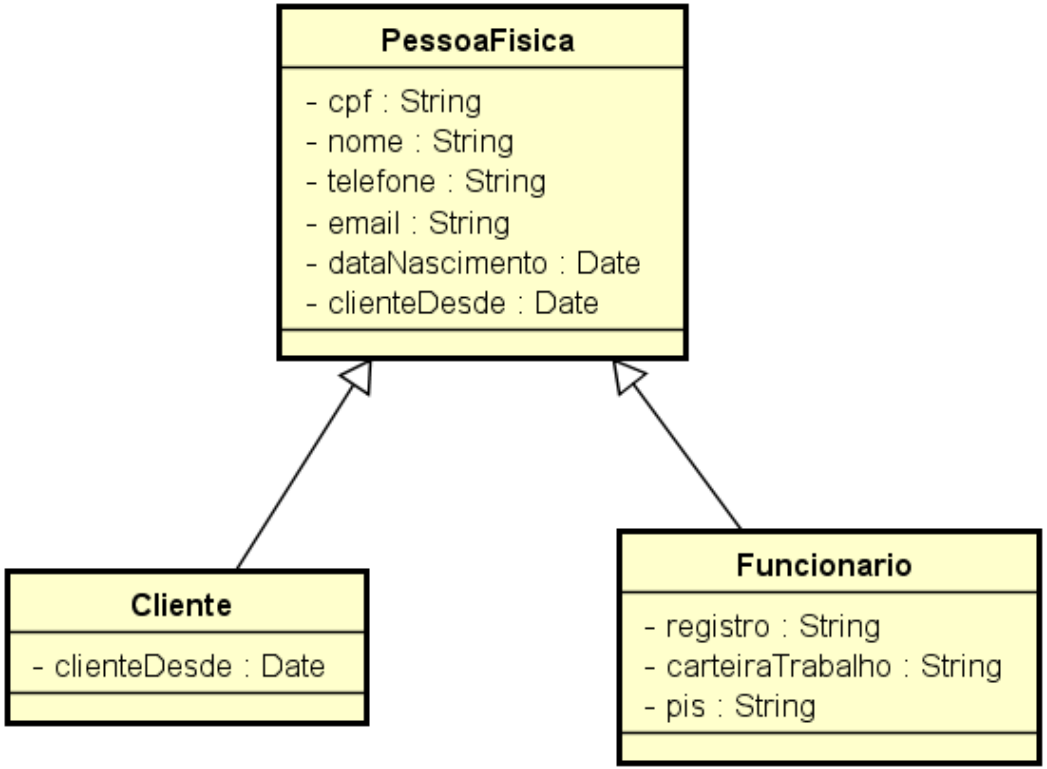
PILARES DO POO



A **herança** permite que uma classe **(subclasse ou classe derivada)** herde os atributos e métodos de outra classe (superclasse ou classe base). A herança promove a reutilização de código, a organização hierárquica e a extensibilidade do sistema.


MÃE
CLASSE BASE
NOME
COR DO OLHO
COR DO CABELO
COR DA PELE
ATRIBUTOS
ANDAR
COMER
DORMIR
MÉTODOS/FUNÇÕES
PAI
HERANÇA
HERANÇA SIMPLES
FILHA
CLASSE FILHA
NOME
COR DO OLHO
COR DO CABELO
COR DA PELE
ATRIBUTOS
COMER
DORMIR
ENGATINHAR
MÉTODOS/FUNÇÕES
HERANÇA MULTIPLA

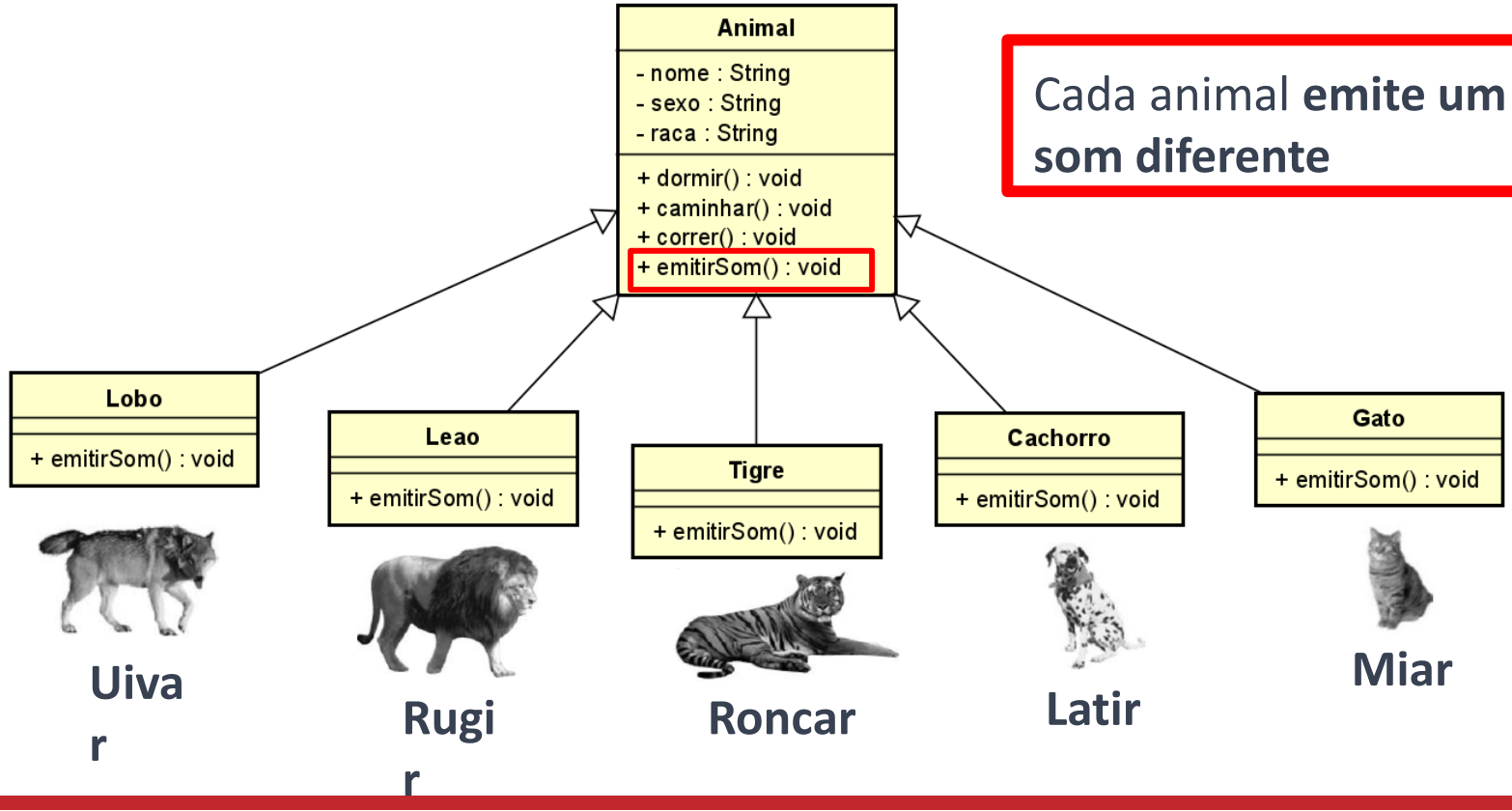
EXEMPLO: DIAGRAMA DE CLASSES



PILARES DO POO

O **polimorfismo** permite que um único método seja usado de maneira diferente com base no objeto que o chama. Isso facilita a flexibilidade e a extensibilidade do código, pois novas classes podem ser adicionadas sem afetar a funcionalidade existente.

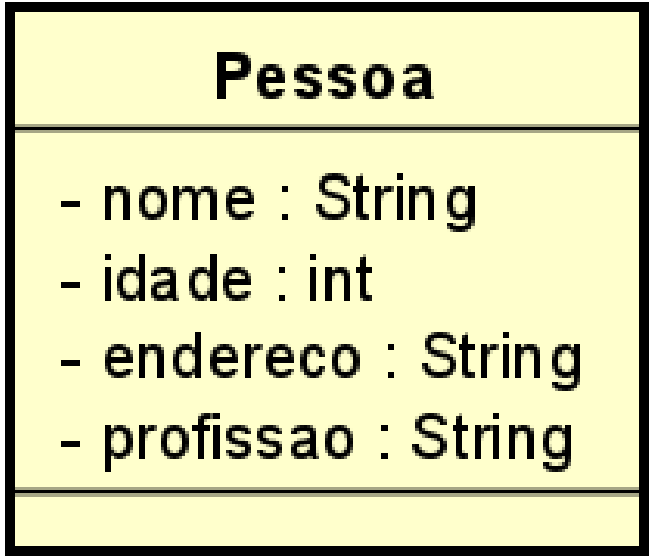
EXEMPLO: POLIMORFISMO



Cada animal **emite um som diferente**

Certo, como implementar em C++?

Diagrama de classe



```cpp
#include <iostream>
#include <string>
using namespace std;
class Pessoa {
private:
    string nome;
    int idade;
    string endereco;
    string profissao;

public:
    // Construtor da classe Pessoa
    Pessoa(string nome, int idade, string endereco, string profissao) {
        this->nome = nome;
        this->idade = idade;
        this->endereco = endereco;
        this->profissao = profissao;
    }

    // Métodos para definir e obter o nome
    void setNome(string novoNome) {
        nome = novoNome;
    }

    string getNome() {
        return nome;
    }

    // Métodos para definir e obter a idade
    void setIdade(int novaIdade) {
        idade = novaIdade;
    }

    string getEndereco() {
        return endereco;
    }

    // Métodos para definir e obter a profissão
    void setProfissao(string novaProfissao) {
        profissao = novaProfissao;
    }

    string getProfissao() {
        return profissao;
    }

    // Método para exibir informações da pessoa
    void mostrarInfo() {
        cout << " " << " " << endl;
        cout << "Nome: " << nome << endl;
        cout << "Idade: " << idade << " anos" << endl;
        cout << "Endereço: " << endereco << endl;
        cout << "Profissão: " << profissao << endl;
    }
};
```

```
int main() {
    // Criando uma instância da classe Pessoa
    Pessoa pessoa1("João", 30, "Rua Principal, 123", "Engenheiro");

    // Exibindo informações da pessoa
    pessoa1.mostrarInfo();

    // Modificando o nome, idade, endereço e profissão da pessoa
    pessoa1.setNome("Maria");
    pessoa1.setIdade(25);
    pessoa1.setEndereco("Avenida Secundária, 456");
    pessoa1.setProfissao("Médica");

    // Exibindo as informações atualizadas da pessoa
    pessoa1.mostrarInfo();

    return 0;
}
```

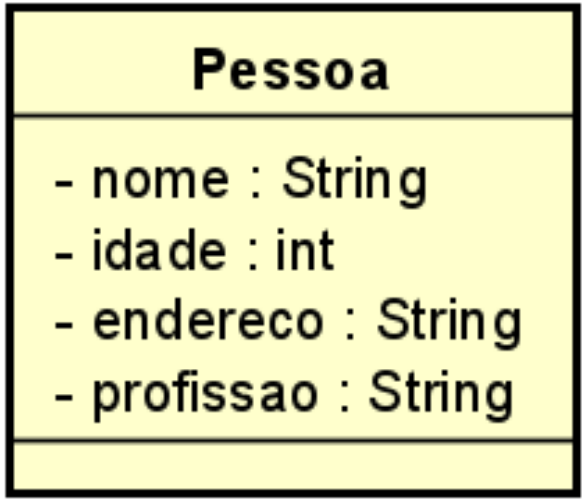
Pessoa 1

João
30
Rua Principal, 123
Engenheiro

Pessoa 1

Maria
25
Av. Secundária, 456
Médica

MODIFIQUE A CLASSE PESSOA, ADICIONANDO OS ATRIBUTOS E MÉTODOS PARA CPF E RG E IMPLEMENTE NA CLASSE PRINCIPAL



CRIE A CLASSE CARRO, E ADICIONE 4 ATRIBUTOS E MÉTODOS DE ACORDO COM A IMAGEM :